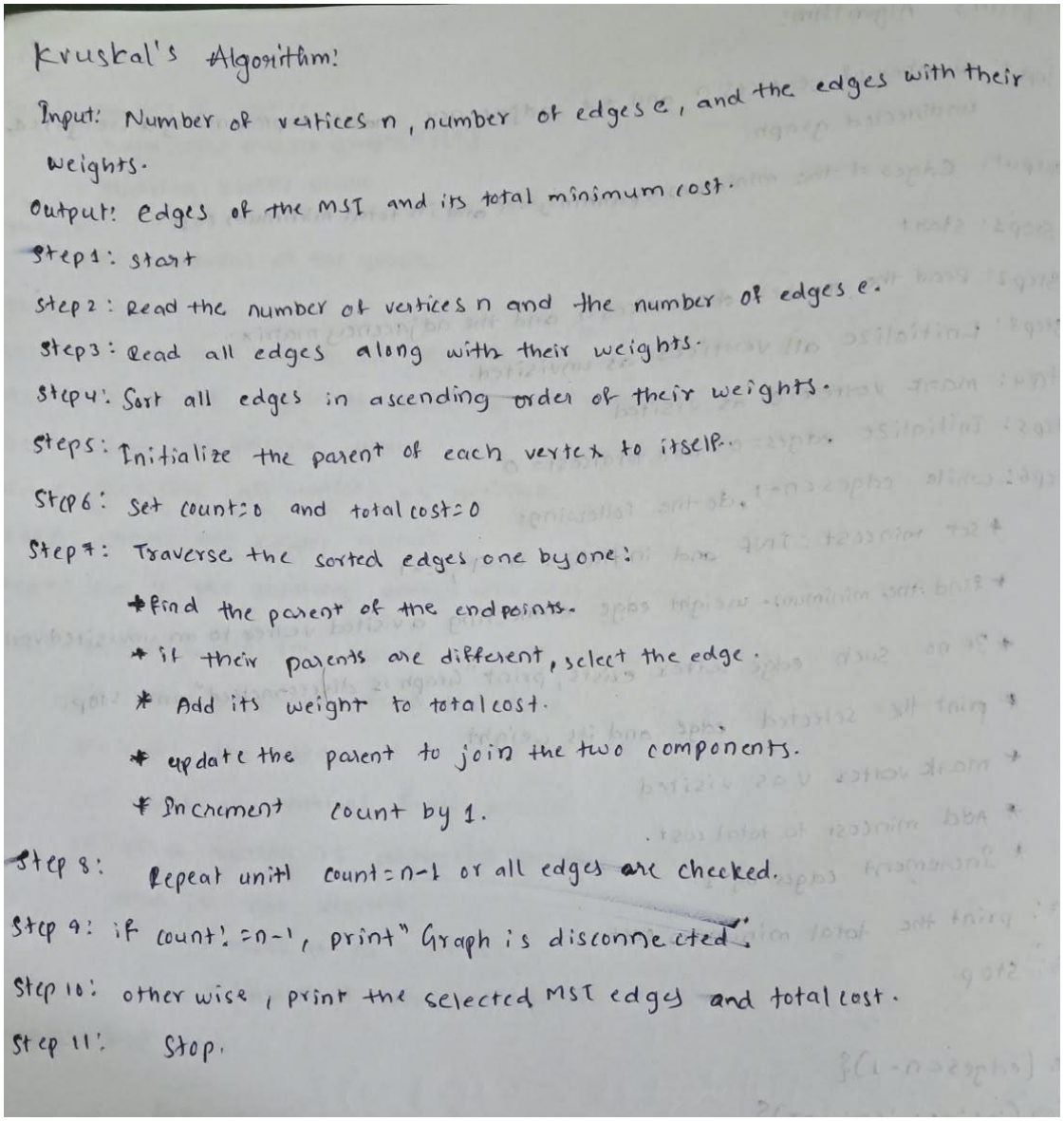
PRACTICAL - 10

AIM :Implementation of prim's algorithm.

Algorithm :



Kruskal's Algorithm:

Input: Number of vertices n, number of edges e, and the edges with their weights.

Output: edges of the MST and its total minimum cost.

Step 1: start

Step 2: Read the number of vertices n and the number of edges e.

Step 3: Read all edges along with their weights.

Step 4: Sort all edges in ascending order of their weights.

Step 5: Initialize the parent of each vertex to itself.

Step 6: Set count=0 and total cost=0

Step 7: Traverse the sorted edges one by one:

* Find the parent of the endpoints.
* if their parents are different, select the edge.
* Add its weight to total cost.
* update the parent to join the two components.
* Increment count by 1.

Step 8: Repeat until count=n-1 or all edges are checked.

Step 9: if count != n-1, print "Graph is disconnected."

Step 10: other wise, print the selected MST edges and total cost.

Step 11: Stop.

code :

```
#include <iostream>
using namespace std;
```

```
struct Edge {
    int u, v, weight;
};

int main() {
    int n, e;
    Edge edge[100];

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter number of edges: ";
    cin >> e;

    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < e; i++) {
        cin >> edge[i].u >> edge[i].v >> edge[i].weight;
    }

    // Sort edges by weight
    for (int i = 0; i < e - 1; i++) {
        for (int j = 0; j < e - i - 1; j++) {
            if (edge[j].weight > edge[j + 1].weight) {
                Edge temp = edge[j];
                edge[j] = edge[j + 1];
                edge[j + 1] = temp;
            }
        }
    }

    int parent[10];

    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }

    int count = 0, totalCost = 0;

    cout << "\nEdges in Minimum Spanning Tree:\n";

    for (int i = 0; i < e && count < n - 1; i++) {
```

```
        int u = edge[i].u;
        int v = edge[i].v;

        int a = u, b = v;

        while (parent[a] != a)
            a = parent[a];

        while (parent[b] != b)
            b = parent[b];

        if (a != b) {
            cout << u << " - " << v
                 << " : " << edge[i].weight << endl;

            parent[a] = b;
            totalCost += edge[i].weight;
            count++;
        }
    }

    if (count != n - 1) {
        cout << "Graph is disconnected!\n";
    } else {
        cout << "Minimum Cost = " << totalCost << endl;
    }

    return 0;
}
```

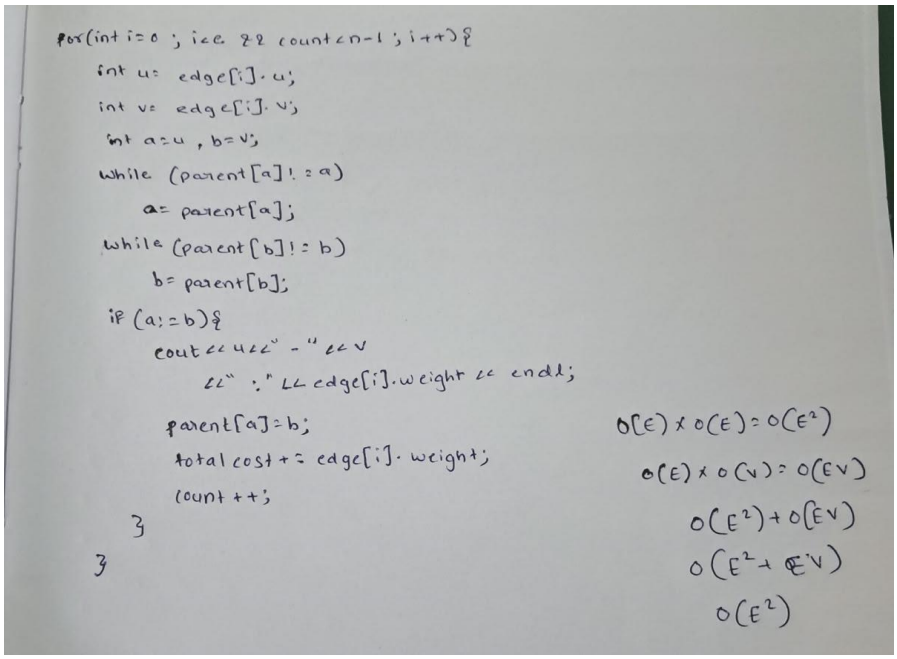
Output :

```
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

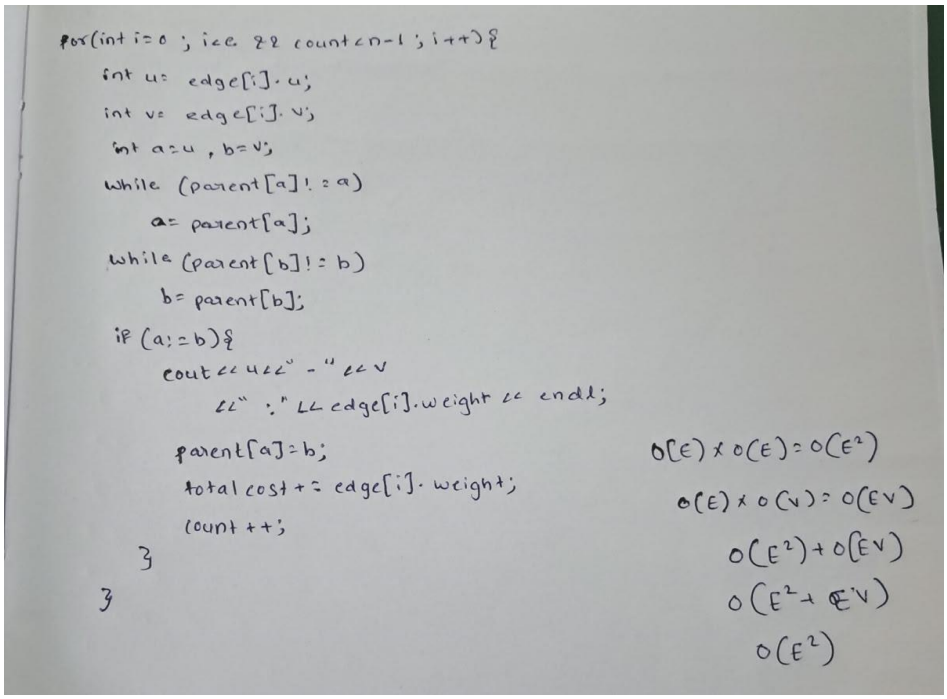
Edges in Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum Cost = 19
```

Time complexity :

Best case :



Average case :



```
for(int i=0 ; i<e && count<n-1 ; i++){
    int u= edge[i].u;
    int v= edge[i].v;
    int a=u , b=v;
    while (parent[a]!=a)
        a= parent[a];
    while (parent[b]!=b)
        b= parent[b];
    if (a!=b){
        cout<<u<<" - "<<v
            <<" :"<<edge[i].weight<<endl;
        parent[a]=b;
        totalcost+= edge[i].weight;
        count++;
    }
}
```

$O(E) \times O(E) = O(E^2)$

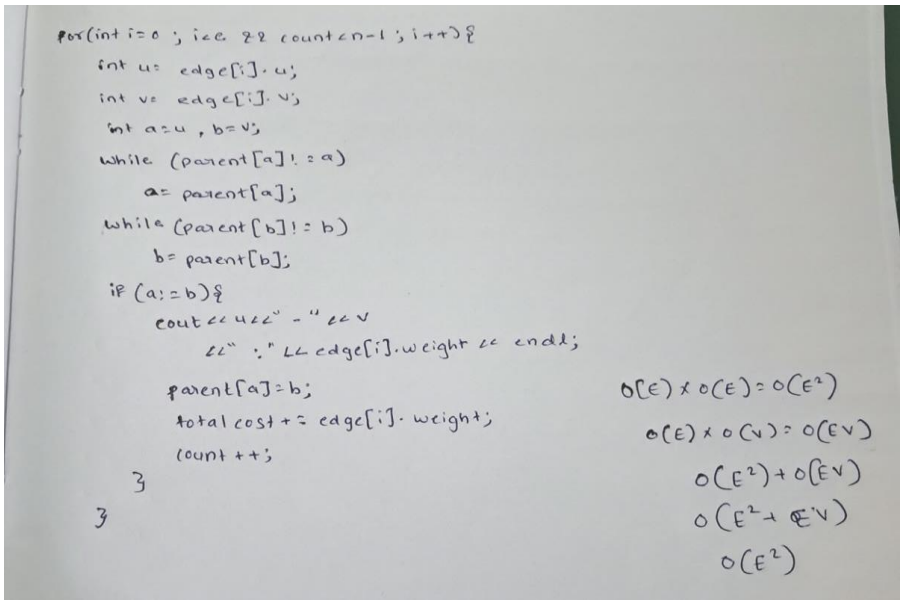
$O(E) \times O(V) = O(EV)$

$O(E^2) + O(EV)$

$O(E^2 + EV)$

$O(E^2)$

Worst case :



```
for(int i=0 ; i<e && count<n-1 ; i++){
    int u= edge[i].u;
    int v= edge[i].v;
    int a=u , b=v;
    while (parent[a]!=a)
        a= parent[a];
    while (parent[b]!=b)
        b= parent[b];
    if (a!=b){
        cout<<u<<" - "<<v
            <<" :"<<edge[i].weight<<endl;
        parent[a]=b;
        totalcost+= edge[i].weight;
        count++;
    }
}
```

$O(E) \times O(E) = O(E^2)$

$O(E) \times O(V) = O(EV)$

$O(E^2) + O(EV)$

$O(E^2 + EV)$

$O(E^2)$

Conclusion:

Kruskal's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph. It sorts all edges in ascending order of weight and selects the minimum-weight edges without forming any cycle. The algorithm continues until

 – *1* edges are selected, producing a spanning tree with minimum total cost. It is useful in network design, road connections, and communication systems.